TECHNICAL SPECIFICATION

Smart Command Architecture: High-Signal Heuristic Scan & AI Lifecycle

A high-efficiency blueprint for developer portfolio automation: combining selective repository heuristic scanning, Sanity CMS context persistence, tiered caching, and synchronized bilingual content generation.

1. Executive Summary & Token Economic Model

Brute-force ingestion of full Git repositories into Large Language Models results in severe token inflation (50,000 to 100,000+ tokens per scan), rate limiting, and prompt dilution caused by boilerplate files. The **Smart Command Architecture (SCA)** adopts an **Intelligent Heuristic Scan** pipeline that isolates high-signal files and developer commit logs, generating rich architectural profiles while reducing token usage by up to **92%**.

Scan Strategy	Data Ingested	Average Tokens	Accuracy & Insights
Full Codebase Ingest	Raw tree, all source code, assets	50,000 – 120,000	High noise, diluted focus, frequent API timeouts
Naive Shallow Scan	Only README.md + package.json	1,500 – 3,000	Prone to boilerplate failure; misses true challenges
SCA: Deep Scan	Tree paths + Configs + Commits	**6,000 – 8,500**	**High fidelity: captures authentic trade-offs & goals**
SCA: Row Refine	Sanity `aiContextDigest`	**600 – 900**	Zero GitHub overhead; synchronizes EN & FA
SCA: Field Polish	Field text + Dossier summary	**250 – 450**	Preserves tone; auto-translates paired counterpart

2. Architectural Foundations

Heuristic Repository Ingest

Uses GitHub REST APIs to extract filtered directory topologies, core package manifests, schema configurations, and author commit logs without cloning full repos.

Persistent Technical Dossier

Gemini distills repo facts into an internal `aiContextDigest` stored in Sanity.io. Subsequent field updates query this cached summary directly.

Bilingual Field Pairing

English (LTR) and Persian (RTL) inputs are unified into coupled UI rows. Updates to one language automatically propagate idiomatic translations to the sibling.

> **Core Principle: Decoupled Extraction and Refinement**
> Expensive operations (network queries and large model prompts) are executed *once* during project onboarding. Day-to-day administrative edits execute lightweight micro-prompts against the persistent digest.

3. The Heuristic Scanner Protocol (`lib/github/scanner.ts`)

The scanner gathers deep project context in four non-destructive API queries. If any component file is missing, the scanner degrades gracefully without interrupting analysis.

`Step A: Topology`	**Recursive Tree API (Filtered):** Strips binaries, vendor code, node_modules, and assets. Leaves clean path topology.
`Step B: Identity`	**Manifests & Documentation:** Reads `package.json` dependencies; evaluates `README.md` against boilerplate templates.
`Step C: Deep Signal`	**Data & Routing:** Whitelist matching for `schema.prisma`, `sanity.config.ts`, `middleware.ts`, and root layout files.
`Step D: Friction Log`	**Commit History API:** Extracts the 15 most recent commit messages to discover true engineering hurdles and bug resolutions.

4. The High-Signal Whitelist Matrix

1. Schema & Data Models

- `prisma/schema.prisma`
- `schemaTypes/*.ts` (Sanity Studio)
- `db/schema.ts` (Drizzle / Kysely)

Value: Reveals business domain entities, relationships, constraints, and state structures without guesswork.

2. Routing, Auth & Infrastructure

- `middleware.ts` (Edge routing, tokens)
- `next.config.js / ts` (Headers, proxies)
- `lib/auth.ts` or `auth.config.ts`

Value: Informs architecture diagrams, security boundaries, and edge computing patterns.

3. Commit History (Engineering Realities)

- `GET /repos/{owner}/{repo}/commits?per_page=15`
- Commit message samples: *"fix: resolve cache race condition"*, *"refactor: migrate to App Router tags"*

Value: Real-world evidence for the "Challenges Overcome" field; prevents fabricated claims.

4. Fallback Detection

- Detects default starter README files.
- Fallback to root UI entrypoint (`app/page.tsx`).
- Synthesizes technical summary from dependencies if descriptions are absent.

Value: Guarantees reliable analysis even for incomplete or poorly documented repos.

5. Sanity Schema Extension (`aiContextDigest`)

Stored as a hidden or read-only document object in Sanity CMS to anchor all micro-generation requests:

> **Fields:** summary (text), architecture (text), coreFeatures (array of string), technicalTradeoffs (array of string), rawDigest (compressed plain-text context), lastAnalyzedCommit (sha), lastAnalyzedAt (datetime).

6. Three-Tier AI Action Hierarchy

The user interface provides three distinct levels of automation, balancing total overhaul with targeted editorial polish.

Action Spectrum Overview

Level 1: Deep Scan (Primary Action) — Full heuristic scan of GitHub repo. Generates the initial baseline metadata, tech stack array, and saves the persistent `aiContextDigest`.

Level 2: Refine (Row Action) — Reads only the stored Sanity dossier. Completely re-extracts both English and Persian field values simultaneously from the same technical ground truth.

Level 3: Polish (Field Action) — Preserves author tone and specific phrasing in the active input. Corrects syntax, enhances professional technical voice, and updates the translation counterpart.

7. Bilingual Localization & LTR Isolation Rules

Persian (FA) Localization Guardrails

- **Idiomatic Phrasing:** Must generate natural developer Persian prose; prohibits literal machine translations.
- **Strict LTR Technical Entities:** All technology names, library tags, CLI commands, file paths, and environment keys must remain in Latin English with `dir="ltr"`.
- **Directional Integrity:** Form inputs enforce CSS text alignment (`text-right` for FA, `text-left` for EN).

Next.js & Edge Caching Tier

- **Public Portfolio (Tier 1):** ISR with Next.js Cache Tags (`tags: ['projects']`). Infinite cache lifetime until an invalidation event occurs.
- **On-Demand Webhook (Tier 2):** Sanity GROQ webhook triggers `/api/revalidate` using HMAC signature verification.
- **Admin Console (Tier 3):** Real-time dynamic fetch (`cache: 'no-store'`) paired with optimistic UI updates.

8. Verification & Security Checklist

Domain	Implementation Requirement	Validation Target
GitHub Rate Limiting	Authenticated Personal Access Token with conditional ETag check	Zero 429 errors; 304 response on unchanged trees
Sanity Invalidation	`next-sanity/webhook` signature validation	Purges public cache tags within <300ms
Model Safety	Strict JSON schema enforcement on Gemini API responses	Zero parse exceptions in admin form binding
RTL Rendering	Typography spans with explicit text-direction attributes	No inverted numbers or broken Latin code snippets